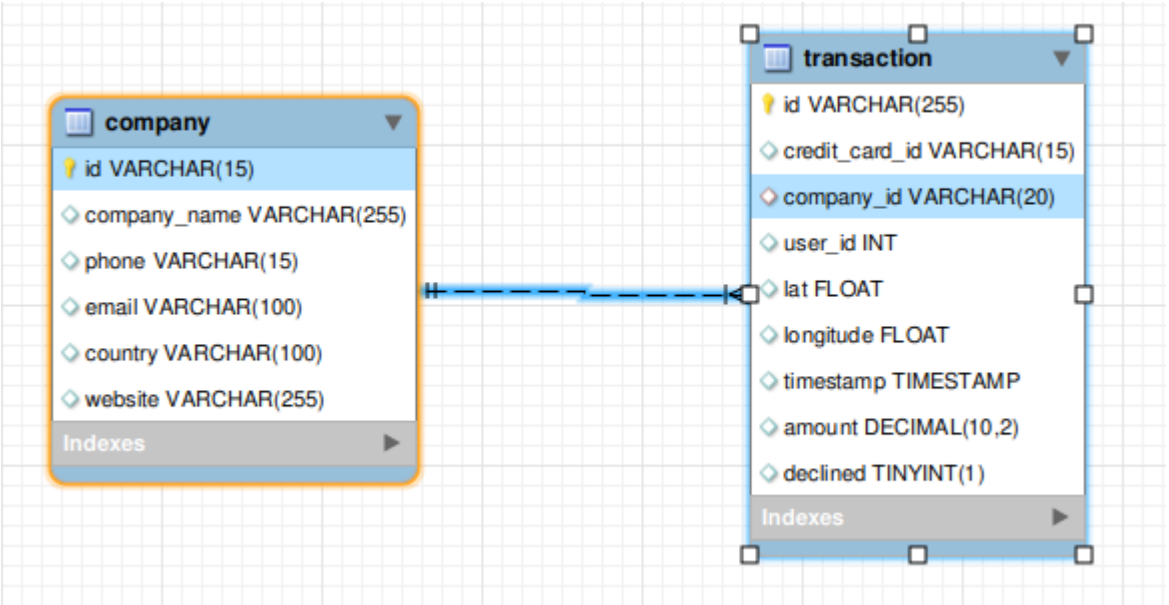
Tasca Sprint 2

1. Nivell 1

Exercici 1
A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.



La tabla 'company' tiene 6 campos y nos da información detallada de las empresas: el nombre de la empresa, teléfono, email, país y web. La PK es 'id'.

La tabla 'transaction' tiene 9 campos y nos da información respecto a las transacciones realizadas por las empresas, es decir, las ventas que hicieron. Encontramos los campos:

- id (PK)
- credit_card_id: identificador de la tarjeta de crédito desde la cual se hizo la transferéncia
- company_id (FK): identificador de la empresa
- user_id: identificador del usuario hizo la transacción
- lat y longitude: las coordenadas desde donde se hizo la transacción o compra
- timestamp: la fecha exacta en la que se hizo
- amount: cantidad de dinero pagada
- declined: indica si la transacción fue aceptada o no

La tabla ‘transaction’ tiene elevada granularidad pues tiene 9 campos para detallar las transacciones, mientras que la tabla ‘company’ tiene menor granularidad ya que solo tiene 6 campos.

La cardinalidad entre tablas es 1:N, es decir, cada empresa de la tabla ‘company’ puede aparecer más de una vez en la tabla ‘transaction’, ya que las empresas pueden hacer varias transacciones.

Finalmente, la tabla transaction representa la tabla de hechos, la cual se vincula con la tabla de dimensiones (company) mediante la FK ‘company_id’.

Exercici 2
Utilitzant JOIN realitzaràs les següents consultes:

Llistat dels països que estan generant vendes.

```
SELECT DISTINCT c.country
FROM `transaction` AS t
JOIN company AS c
ON t.company_id = c.id
WHERE declined = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	country
1	Netherlands
2	Sweden
3	Ireland
4	United States
5	Belgium
6	Canada
7	Germany
8	Norway
9	France
10	Italy
11	United Kingdom
12	New Zealand
13	China
14	Spain
15	Australia

Result 21

Action Output

#	Time	Action	Message
1	18:54:09	SELECT DISTINCT c.country FROM `transaction` AS t JOIN company AS c ON t.company_id = c.id WHERE declined = 0	16 row(s) returned

Hago una join entre las dos tablas con el filtro WHERE con declined =0 para ver sólo las ventas realizadas. En el select uso DISTINC para obtener países diferentes.

Des de quants països es generen les vendes.

```
14 • SELECT count(DISTINCT c.country) AS total_paises
15   FROM `transaction` AS t
16   JOIN company AS c
17   ON t.company_id = c.id
18   WHERE declined = 0;
19
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	total_paises
1	15

Result 22

Action Output

#	Time	Action	Message
1	18:55:04	SELECT count(DISTINCT c.country) AS total_paises FROM `transaction` AS t JOIN company AS c ON t.company_id = c.id WHERE declined ...	1 row(s) returned

La misma query que antes pero ahora añado el count para contar los países que generan ventas.

Identifica la companyia amb la mitjana més gran de vendes.

```
SELECT c.id, company_name, round(avg(amount),2) as avg_vendes
FROM `transaction` AS t
JOIN company AS c
ON t.company_id = c.id
GROUP BY c.id, company_name
ORDER BY round(avg(amount),2)
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	id	company_name	avg_vendes
1	b-2222	Ac Fermentum Incorporated	284.87

Action Output

#	Time	Action
1	17:10:10	SELECT c.id, company_name, round(avg(amount),2) as avg_vendes FROM transaction AS t JOI...

Hago una join, luego agrupo por empresa, calculo la media redondeando a 2 decimales, ordeno por la media de mayor a menor finalmente me quedo con el mayor al aplicar LIMIT 1.

Exercici 3
Utilitzant només subconsultes (sense utilitzar JOIN):

Mostra totes les transaccions realitzades per empreses d'Alemanya.

```
SELECT t.id AS transaccion, t.company_id AS empresa
FROM `transaction` AS t
WHERE declined = 0
AND
EXISTS (
    SELECT id FROM company AS c
    WHERE t.company_id = c.id AND
    country = 'Germany'
    );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

#	transaccion	empresa
1	00138D3B-206D-4C03-94B7-63A2...	b-2222
2	0013C1B6-3B84-4D6C-8154-E2B...	b-2222
3	00201A11-2E62-44C4-941D-198F...	b-2222
4	00235618-0A5C-4D49-9DCB-B3A...	b-2222
5	005A5A7B-1F1A-4B6C-9B15-162...	b-2222
6	00687139-48B2-4FFA-8E73-B203...	b-2222
7	0074F4DD-32F1-4827-8758-5589...	b-2222
8	00AAB9CD-39D6-4DCB-8A1D-13...	b-2222
9	00BE09D4-6920-47D8-ABE8-325...	b-2222
10	00DA0383-E048-4577-8ED1-3C56...	b-2222
11	00DD11DE-ED01-4BBD-93A0-17...	b-2222

transaction 19

Action Output

#	Time	Action	Message
1	18:53:14	SELECT t.id AS transaccion, t.company_id AS empresa FROM `transaction` AS t WHERE declined = 0 AND EXISTS (SELECT id FROM co...	13269 row(s) returned

Hago una subconsulta con EXISTS de manera que me devuelve las empresas cuyo id de company hace match con el company_id de 'transactions' estableciendo el filtro por el país. También con declined = 0 para obtener las transacciones aceptadas.

Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

```
SELECT DISTINCT t.company_id AS empresa
FROM `transaction` AS t
WHERE t.amount > (
                SELECT AVG(amount)
                FROM `transaction` WHERE declined=0)
AND declined = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	empresa
1	b-2222
2	b-2226
3	b-2230
4	b-2234
5	b-2238
6	b-2242
7	b-2246
8	b-2250
9	b-2254
10	b-2258
11	b-2262
12	b-2266
13	b-2270

transaction 28

Action Output

#	Time	Action	Message
1	19:10:40	SELECT DISTINCT t.company_id AS empresa FROM `transaction` AS t WHERE t.amount > (SELECT AVG(amount) FRO...	100 row(s) returned

Hago una subquery donde se muestran las empresas cuyo amount es mayor a la media de amount de todas las transacciones. Aplico el filtro WHERE tanto en la subquery como fuera para que me considere solo las transacciones aceptadas.

Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

```
54 • SELECT id, company_name
55   FROM company AS c
56   WHERE NOT EXISTS
57       (SELECT company_id
58       FROM `transaction` AS t
59       WHERE c.id = t.company_id
60       AND declined= 0);
61
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

#	id	company_name
*	NULL	NULL

Action Output

#	Time	Action	Message
1	19:11:48	SELECT id, company_name FROM company AS c WHERE NOT EXISTS (SELECT company_id FROM `transaction` AS t WHERE c.id ...	0 row(s) returned

Hago una subquery con NOT EXISTS de manera que se muestran aquellas empresas que no tienen transacciones porque el id de company no hace match con el company_id de transaction.

Exercici 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

```
72 • CREATE TABLE IF NOT EXISTS credit_card(
73       id VARCHAR (30),
74       iban VARCHAR (50),
75       pan VARCHAR(50),
76       pin CHAR(5),
77       cvv CHAR(4),
78       expiring_date VARCHAR (10),
79       PRIMARY KEY (id)
80   );
81
```

Action Output

#	Time	Action	Message
1	19:45:38	CREATE TABLE IF NOT EXISTS credit_card(id VARCHAR (30), iban VARCHAR (50), pan VARCHAR(50), pin CHAR(5), cvv CHAR(4), ...	0 row(s) affected

```
7  • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT8780622813509242945634 6', '544 58654 54343 384
8  • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2980', 'DE39241881883086277136', '402400 7145845969', '5
9  • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2987', 'GE896814348377487818 13', '3763 747687 76666', '2
10 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', 'BH6271442836806676529 4', '344283273252593', '754
11 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CY49087426654774581266832110', '511722 924833 22
12 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU507216693616119230', '4485744464433884', '1856
13 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS11939821629571596834245 6821', '3784 662233 173
14 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 1379 4842 3
15 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ62317413982441418123739746', '3429 279566 7763
16 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3036', 'AZ39336002925842865843941994', '3768 451556 4876
17 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3043', 'TN64881433105148521795 35', '455676 6437463635',
18 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3050', 'FR5167744369175836831854477', '4024007123722', '
19 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3057', 'LU931822574697545215', '3484 621767 21237', '680
20 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3064', 'PS14696554544925337762727313 3', '3467 732741 268
21 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3071', 'N08923814763512', '3464 789562 23352', '6625', '
22 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3078', 'IS0251271458846232795487 33', '4539 322 74 2377',
23 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3085', 'BE63114723972437', '5266 3346 1135 1687', '7241'
24 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3092', 'RO65LSOD1166122125447487', '3488 754223 46253',
```

Action Output

#	Time	Action	Message
4995	19:47:32	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9576', 'XX5297109305877126417 2007', '455422697806410...	1 row(s) affected
4996	19:47:32	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9577', 'XX1589140785948086331 47086', '81104372718561...	1 row(s) affected
4997	19:47:32	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9578', 'XX9915396464561105678 70254', '89998088230614...	1 row(s) affected
4998	19:47:32	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX2963930915871702021 31236', '96900604686786...	1 row(s) affected
4999	19:47:32	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX7812588898519508066 77358', '55411823644989...	1 row(s) affected
5000	19:47:32	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX9156705164053881243 98147', '26243054701676...	1 row(s) affected

```
82 • SELECT * FROM credit_card;
83
84
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

#	id	iban	pan	pin	cvv	expiring_date
1	CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
2	CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
3	CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	11/26/26
4	CcS-4860	XX5559590368835304645299	2481155515498459	6876	661	07/27/27
5	CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	04/25/26
6	CcS-4862	XX4774721462463645409758	6715617009807829	4042	174	11/27/26
7	CcS-4863	XX1476829664245046207111	3140879819451394	5969	449	12/27/29
8	CcS-4864	XX8380298893385731196159	5793672133649114	8481	139	02/28/26
9	CcS-4865	XX7085078596101025280599	5101552687251312	7847	903	11/25/28
10	CcS-4866	XX4792859188206596406839	8080768801072613	9271	961	02/28/25
11	CcS-4867	XX6038298816319374853717	7761849537661098	4820	862	11/30/25
12	CcS-4868	XX3929101617300533068044	4160419663151801	6860	549	08/25/25
13	CcS-4869	XX1566712096111531886465	5892930716233310	4775	448	09/29/25
14	CcS-4870	XX6443663804167732133949	2227240879956178	5872	715	01/25/25
15	CcS-4871	XX3520209528376786066563	6944143455556838	7992	643	07/26/26
16	CcS-4872	XX4091621143155817546334	5035418997919459	3642	191	12/29/26

credit_card 33

Action Output

#	Time	Action	Message
1	19:49:51	SELECT * FROM credit_card	5000 row(s) returned

Primero creo la tabla con CREATE TABLE estableciendo los distintos campos y el tipo de dato así como la PK. Para conocer el tipo de dato y su extensión, primero abrí la tabla en el terminal de Ubuntu. Después cargue los datos y comprobé que se cargaron bien.

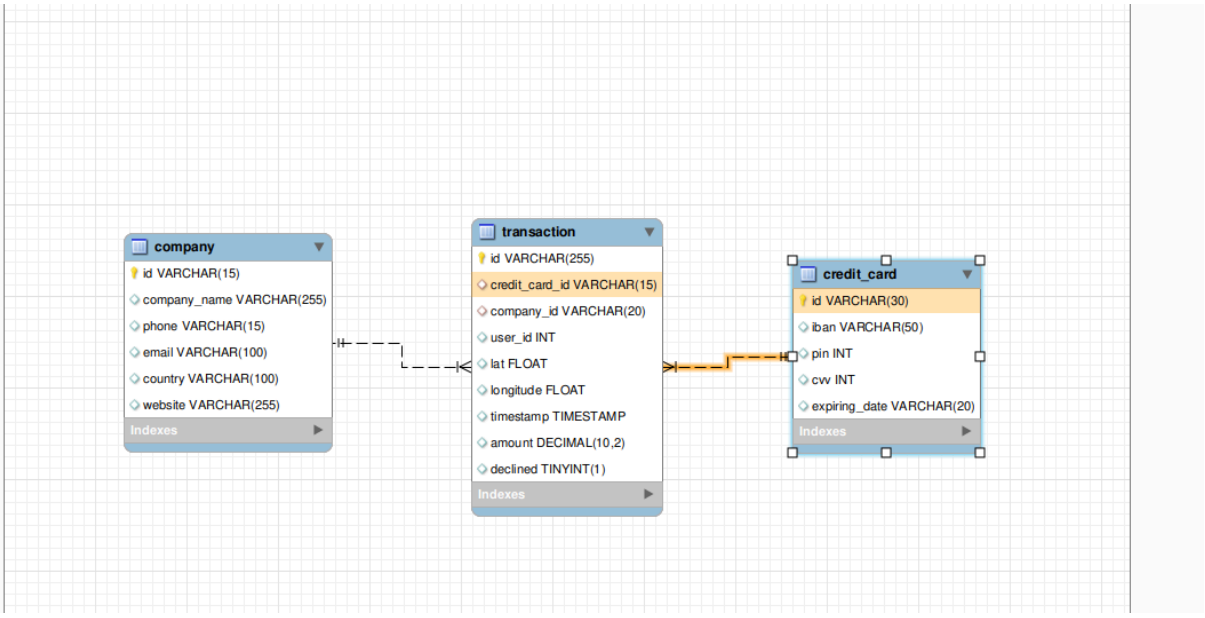
```
82 • UPDATE credit_card
83   SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')
84   LIMIT 6000;
85
```

Action Output

#	Time	Action	Message
1	20:45:52	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y') LIMIT 6000	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0

Finalmente, modifique el tipo de dato de la columna de las fechas con UPDATE / SET y con la función STR_TO DATE.

Generé la FK y la relación en el esquema. Existe una opción para ello y me parece muy util e intuitivo.



La tabla de transaction es la de hechos mientras que company y credit_card representan dimensiones.

Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

```
99 •  UPDATE credit_card SET iban = 'TR323456312213576817699999'
100   WHERE id = 'CcU-2938';
101
102 •  SELECT * FROM credit_card
103   WHERE id = 'CcU-2938';
104
105
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

#	id	iban	pan	pin	cvv	expiring_date
1	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30
*	NULL	NULL	NULL	NULL	NULL	NULL

Action Output

	#	Time	Action	Message
✓	1	21:27:56	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	2	21:28:01	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned

Con update/set puede actualizar la columna de interés y con where, el registro en cuestión.

Exercici 6
En la taula "transaction" ingressa una nova transacció amb la següent informació:

```
Id

108B1D1D-5B23-A76C-55EF-C568E49A99DD

credit_card_id

CcU-9999

company_id

b-9999

user_id

9999

lat

829.999

longitude

-117.999

amount

111.11

declined

0
```

```
110 •  INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)
111    VALUEs ('108B1D1D-5B23-A76C-55EF-C568E49A99DD','CcU-9999', 'b-9999', '9999', '829.999', '-117.999', current_timestamp(), '111.11', '0');
112
113
```

Action Output

#	Time	Action	Message
1	19:01:33	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUEs ('108B1D1D-5B...	1 row(s) affected

Con INSERT INTO añado la nueva transacción con los datos facilitados. En la columna timestamp añadí la hora actual con CURRENT_TIMESTAMP(), es decir, la hora en la que inserté los valores.

Exercici 7
Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

```
116
117 •  ALTER TABLE credit_card
118        DROP COLUMN pan;
119
```

Action Output

#	Time	Action	Message
1	19:03:01	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings

Con ALTER TABLE nombre_tabla y DROP COLUMN nombre_columna, elimino la columna pan.

Exercici 8
Descarrega els arxius CSV que trobaràs a l'apartat de recursos:

american_users.csv
european_users.csv
companies.csv
credit_cards.csv
transactions.csv
Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Primero creo la base de datos:

```
125        -- Exercici 8
126    -- creo la base de dades
127 •  CREATE DATABASE mi_negocio;
128 •  USE mi_negocio;
129
```

Action Output

#	Time	Action
1	22:14:37	CREATE DATABASE mi_negocio
2	22:14:43	USE mi_negocio

A continuación, para tener una idea de las tablas y poder hacer la creación y configuración, observé las primeras filas de cada una de las tablas en el terminal (aquí muestro un ejemplo de la tabla european_users) :

```
> head -n5 'N1.Ex.8__ european_users.csv'
id,name,surname,phone,email,birth_date,country,city,postal_code,address
151,Meghan,Hayden,0800 746 6747,arcu.vel@hotmail.ca,"Jul 2, 1980",United Kingdom,London,EC1A 1BB,Ap #432-4493 Aliquet Rd.
152,Hakeem,Alford,(0111) 367 0184,adipiscing.ligula@google.edu,"Sep 30, 1979",United Kingdom,Birmingham,B1 1AA,551-8930 Lobortis Street
153,Keegan,Pugh,(016977) 3851,sodales.nisi@aol.org,"Jul 27, 1994",United Kingdom,London,EC1A 1BB,Ap #312-5898 Consectetuer St.
154,Cooper,Bullock,(021) 2521 6627,et@outlook.net,"Nov 2, 1986",United Kingdom,Manchester,M1 1AE,872-1866 Pede Rd.
```

Luego fui creando cada tabla, dejando para el final la tabla de transactions, pues es la de hechos. Cuando creo las tablas ya configuro la PK, pero las FK las establezco más tarde.

```sql
CREATE TABLE american_users(
    id INT PRIMARY KEY,
    name VARCHAR (25),
    surname VARCHAR (50),
    phone VARCHAR (25),
    email VARCHAR(50),
    birth_date VARCHAR(22),
    country VARCHAR (30),
    city VARCHAR(30),
    postal_code VARCHAR(25),
    address VARCHAR (50)
    );
```

Action Output

#	Time	Action	Message
1	22:25:54	CREATE TABLE american_users(id INT PRIMARY KEY, name VARCHAR (25), surname VARCHAR (50), phone VARCHAR (25), ema...	0 row(s) affected

```sql
CREATE TABLE european_users(
id INT PRIMARY KEY,
name VARCHAR (25),
surname VARCHAR (50),
phone VARCHAR (25),
email VARCHAR(50),
birth_date VARCHAR(22),
country VARCHAR (30),
city VARCHAR(30),
postal_code VARCHAR(25),
address VARCHAR (50)
);
```

Action Output

#	Time	Action	Message
1	22:27:41	CREATE TABLE european_users(id INT PRIMARY KEY, name VARCHAR (25), surname VARCHAR (50), phone VARCHAR (25), email VARCH...	0 row(s) affected

```sql
CREATE TABLE companies (
    company_id VARCHAR (8) NOT NULL,
    company_name VARCHAR (40),
    phone VARCHAR(15),
    email VARCHAR(70),
    country TEXT(30),
    website VARCHAR(50),
    PRIMARY KEY (company_id)
    );
```

Action Output

#	Time	Action	Message
1	22:29:41	CREATE TABLE companies (company_id VARCHAR (8) NOT NULL, company_name VARCHAR (40), phone VARCHAR(15), email V...	0 row(s) affected

```sql
CREATE TABLE credit_cards (
    id VARCHAR(10) PRIMARY KEY,
    user_id INT,
    iban VARCHAR(50),
    pan VARCHAR(20),
    pin SMALLINT,
    cvv SMALLINT,
    track1 VARCHAR(100),
    track2 VARCHAR(100),
    expiring_date VARCHAR(10)
    );
```

Action Output

#	Time	Action	Message
1	22:31:06	CREATE TABLE credit_cards (id VARCHAR(10) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin SMALLI...	0 row(s) affected

```sql
227 • CREATE TABLE transactions (
228       id VARCHAR(255) NOT NULL,
229       card_id VARCHAR(10),
230       business_id VARCHAR(8),
231       timestamp TIMESTAMP,
232       amount DECIMAL (10,2),
233       declined TINYINT,
234       product_ids VARCHAR (50),
235       user_id INT,
236       lat FLOAT(24),
237       longitude FLOAT(24),
238       PRIMARY KEY (id)
239       );
240
241   -- Carrego les dades i comprovo que s'han afegit bé
242 •  SELECT * FROM transactions;
243
```

Action Output

#	Time	Action	Message
1	22:32:00	CREATE TABLE transactions (id VARCHAR(255) NOT NULL, card_id VARCHAR(10), business_id VARCHAR(8), timestamp TIM...	0 row(s) affected

Una vez tengo las tablas creadas, cargo los datos desde el mysql pero en mi terminal porque me estaba dando problemas para configurar infile=1 en workbench.

Primero accedo a mysql.

```
> mysql --local-infile=1 -u root -p mi_negocio
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 85
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Luego cargo los datos csv con el siguiente comando:

LOAD DATA LOCAL INFILE 'ruta_archivo'
INTO TABLE tabla_destino
FIELDS TERMINATED BY ',' (separador de campos)
ENCLOSED BY "" (campos entre comillas)
LINES TERMINATED BY '\n' (salto de línea)
IGNORE 1 ROWS; (ignora la primera fila que corresponden al título campo)

En la tabla transactions, el separador es ; por lo que hay que tener en cuenta este aspecto. Las demás tablas se cargan igual, simplemente modificando la ruta y la tabla de destino.

```
    -> LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1-Ex.8__ american_users.csv'ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t
hat corresponds to your MySQL server version for the right syntax to use near 'ignora la primera fila que corresponden al titulo campo)
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1-Ex.8__ american_users.csv' INTO TABLE american_users FIELDS TERMINATED BY ','  ENCLOSED BY '"'  LINES
 TERMINATED BY '\n'  IGNORE 1 ROWS;
Query OK, 1010 rows affected (0,11 sec)OSED BY '"'  LINES TERMINATED BY '\n'  IGNORE 1 ROWS;
Records: 1010  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ european_users.csv' INTO TABLE european_users FIELDS TERMINATED BY ','  ENCLOSED BY '"'  LINES
 TERMINATED BY '\n'  IGNORE 1 ROWS;
Query OK, 3990 rows affected (0,42 sec)
Records: 3990  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ companies.csv' INTO TABLE companies FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATE
D BY '\n' IGNORE 1 ROWS;
Query OK, 100 rows affected (0,02 sec)
Records: 100  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ credit_cards.csv' INTO TABLE credit_cards FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TER
MINATED BY '\n' IGNORE 1 ROWS;
Query OK, 5000 rows affected (0,43 sec)
Records: 5000  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ transactions.csv' INTO TABLE transactions FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TER
MINATED BY '\n' IGNORE 1 ROWS;
Query OK, 100000 rows affected (1,64 sec)
Records: 100000  Deleted: 0  Skipped: 0  Warnings: 0
```

Después de confirmar que se cargaron todos los datos correctamente, decidí crear la tabla users para unir la tabla american_users y european_users, pues ambas tenían los mismos campos y me resultaba más fácil hacer las relaciones.

No obstante, con el objetivo de no perder ninguna información, creé la columna continent en ambas tablas para colocar el continente al cual pertenecen. Como la actualización es para todos los registros, hice UPDATE/ SET sin WHERE y añadí el LIMIT por tema de permisos.

```
177 •  ALTER TABLE european_users
178        ADD continent CHAR (10);
179
180 •  UPDATE european_users
181        SET continent = 'europe'
182        LIMIT 5000;
183
184 •  SELECT * FROM european_users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

#	id	name	surname	phone	email	birth_date	country	city	postal_code	address	continent
1	151	Meghan	Hayden	0800 746 6747	arcu.vel@hotmail.ca	Jul 2, 1980	United Kingdom	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.	europe
2	152	Hakeem	Alford	(0111) 367 0184	adipiscing.ligula@google.edu	Sep 30, 1979	United Kingdom	Birmingham	B1 1AA	551-8930 Lobortis Street	europe
3	153	Keegan	Pugh	(016977) 3851	sodales.nisi@aol.org	Jul 27, 1994	United Kingdom	London	EC1A 1BB	Ap #312-5898 Consectetuer St.	europe
4	154	Cooper	Bullock	(021) 2521 6627	et@outlook.net	Nov 2, 1986	United Kingdom	Manchester	M1 1AE	872-1866 Pede Rd.	europe
5	155	Joshua	Russell	055 4409 5286	justo.nec.ante@outlook.edu	Jan 23, 1984	United Kingdom	Manchester	M1 1AE	Ap #285-4727 Auctor. Av.	europe
6	156	Remedios	Case	055 3114 1566	mollis.phasellus.libero@aol.com	Oct 9, 1994	United Kingdom	Liverpool	L1 1AA	479-3690 Turpis Road	europe
7	157	Philip	Carey	0800 640 6251	phasellus@yahoo.net	Oct 10, 1992	United Kingdom	Manchester	M1 1AE	196-1103 Quisque Street	europe
8	158	Fatima	Dyer	0800 1111	adipiscing@google.org	Dec 24, 1988	United Kingdom	Birmingham	B1 1AA	Ap #435-7194 Scelerisque, St.	europe
9	159	Kylynn	Acevedo	056 5727 9602	dignissim.lacus.aliquam@google.org	May 10, 2000	United Kingdom	Liverpool	L1 1AA	871-3506 Lectus. Ave	europe

european_users 59

Action Output

#	Time	Action	Message
1	23:02:48	UPDATE european_users SET continent = 'europe' LIMIT 5000	3990 row(s) affected Rows matched: 3990 Changed: 3990 Warnings: 0
2	23:03:03	SELECT * FROM european_users	3990 row(s) returned

```
186 • ALTER TABLE american_users
187       ADD continent CHAR (10);
188
189 • UPDATE american_users
190       SET continent = 'america'
191       LIMIT 5000;
192
193 • SELECT * FROM american_users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

#	id	name	surname	phone	email	birth_date	country	city	postal_code	address	continen
1	1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	United States	New York	10001	348-7818 Sagittis St.	america
2	2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	Aug 23, 1992	United States	Philadelphia	19101	903 Sit Ave	america
3	3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	Apr 29, 1998	United States	Houston	77001	736-2063 Tellus St.	america
4	4	Howard	Stafford	1-411-740-3269	ornare.egestas@icloud.edu	Feb 18, 1989	United States	Phoenix	85001	Ap #545-2244 Erat. Rd.	america
5	5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	Sep 26, 1998	United States	Philadelphia	19101	341-2821 Ultrices Av.	america
6	6	Joel	Tyson	(718) 288-8020	gravida.nunc.sed@yahoo.ca	Oct 15, 1989	United States	San Jose	95101	888-2799 Amet Street	america
7	7	Rafael	Jimenez	(817) 689-0478	eget@outlook.ca	Dec 4, 1981	United States	Chicago	60601	8627 Malesuada Rd.	america
8	8	Nissim	Franks	(692) 157-3469	egestas.aliquam.fringilla@google.ca	Aug 1, 1993	United States	New York	10001	Ap #251-7144 Integer St.	america
9	9	Mannix	Mcclain	(590) 883-2184	aliquam.nisl@outlook.com	Jan 24, 1987	United States	San Antonio	78201	647-3080 Lacus. St.	america

american_users 61

Action Output

#	Time	Action	Message
1	23:05:12	ALTER TABLE american_users ADD continent CHAR (10)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	23:05:19	UPDATE american_users SET continent = 'america' LIMIT 5000	1010 row(s) affected Rows matched: 1010 Changed: 1010 Warnings: 0
3	23:05:25	SELECT * FROM european_users	3990 row(s) returned
4	23:05:40	SELECT * FROM american_users	1010 row(s) returned

```
197 • CREATE TABLE users AS
198       SELECT * FROM american_users
199       UNION
200       SELECT * FROM european_users;
201
202 • ALTER TABLE users
203   ADD PRIMARY KEY (id);
204
```

Action Output

#	Time	Action	Message
1	23:08:20	CREATE TABLE users AS SELECT * FROM american_users UNION SELECT * FROM european_users	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
2	23:08:43	ALTER TABLE users ADD PRIMARY KEY (id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Una vez creado el mismo campo en ambas tablas y rellenado con el continente, hice la unión en la tabla users y a continuación especifiqué la PK.

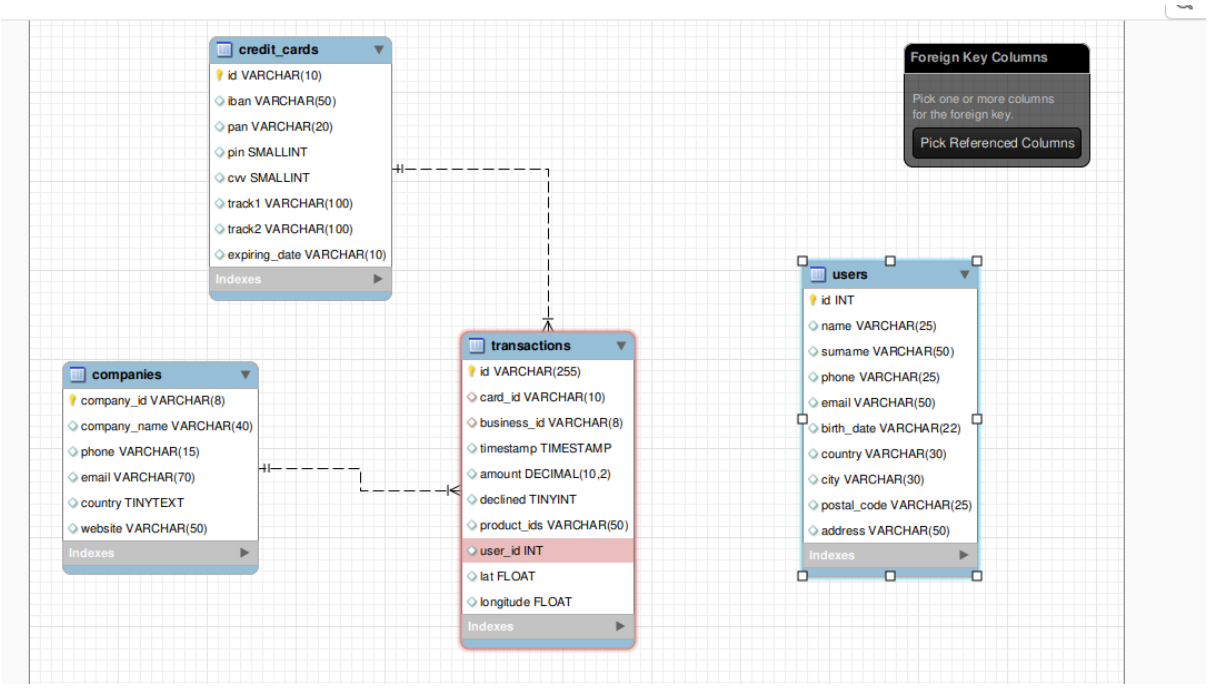
Finalmente, comprobé que la tabla final estaba bien y borré las tablas originales.

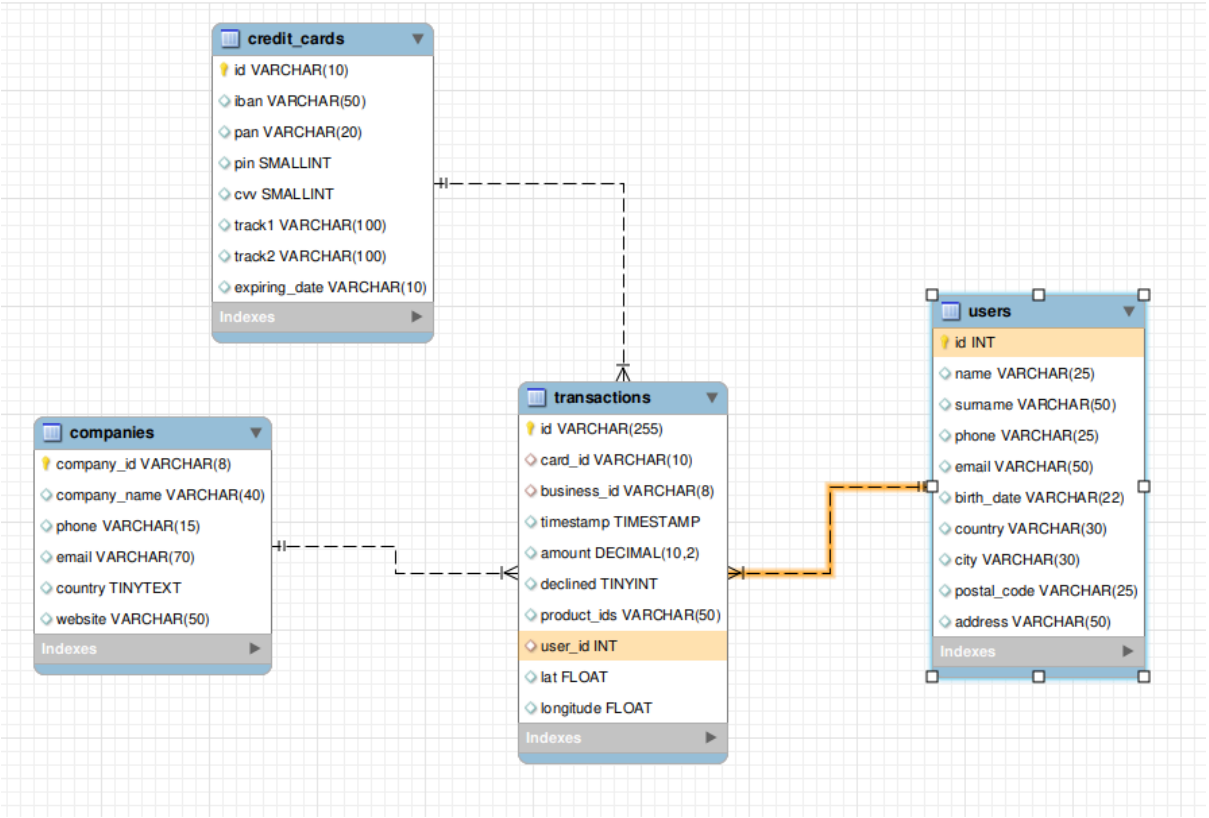
```
205 • SELECT * FROM users;
206
207   -- BORRO les taules american_users i european_users
208 • DROP TABLE american_users;
209 • DROP TABLE european_users;
210
```

Action Output

#	Time	Action
1	23:10:53	SELECT * FROM users
2	23:11:03	DROP TABLE american_users
3	23:11:05	DROP TABLE european_users

Para la creación del esquema, configuré las FK desde la ventana del esquema mismo (me resulta más fácil e intuitivo) y finalmente junté la tabla users con la de transactions, formando el modelo en estrella.





Exercici 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
255 •  SELECT u.id, `name`, surname, count(*) n_trans
256    FROM transactions AS t
257    JOIN users AS u
258    ON u.id = t.user_id
259    WHERE t.declined = 0
260    GROUP BY u.id, `name`, surname
261    HAVING n_trans > 80
262    ORDER BY count(*) DESC;
263
264
265        -- Exercici 10
266    -- Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	id	name	surname	n_trans
1	185	Molly	Gilliam	107
2	289	Dxwgi	Hwcru	91
3	318	Bnyr	Astuw	86

Result 68

Action Output

#	Time	Action	Message
1	23:53:17	SELECT u.id, `name`, surname, count(*) n_trans FROM transactions AS t JOIN users AS u ON u.id = t.user_id WHERE t.declined = 0 GROUP...	3 row(s) returned

Hice una join, luego agrupo por usuario, hago el count(*) para mirar el número de transacciones con WHERE declined = 0 para obtener las aceptadas y finalmente con el having muestro aquellos usuarios con transacciones superiores a 80.

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
268 •  SELECT cc.iban, ROUND (avg(t.amount),2) AS media
269    FROM transactions AS t
270    JOIN companies AS c
271    ON t.business_id = c.company_id
272    JOIN credit_cards AS cc
273    ON cc.id = t. card_id
274    WHERE company_name = 'Donec Ltd' AND
275    t.declined = 0
276    GROUP BY cc.iban
277    ORDER BY ROUND (avg(amount),2) DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	iban	media
1	XX3830178139196201993663 52	680.69
2	XX6377063573975703949739 13	680.01
3	XX9713939714652922023122 59	645.46
4	XX1718471169288923759693 07	628.89
5	XX2254246388185424062235 75	608.68
6	XX7488907290571957117660 71	607.29
7	TN9614563570667381893122	605.41

Result 73

Action Output

#	Time	Action	Message
1	00:00:36	SELECT cc.iban, ROUND (avg(t.amount),2) AS media FROM transactions AS t JOIN companies AS c ON t.business_id = c.company_id JOIN...	370 row(s) returned

Hago una join, con WHERE selecciono la empresa en cuestión y las transacciones aceptadas, luego agrupo por iban y con la función de agregación AVG calculo la media de amount.

2. Nivell 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```
7  SELECT *
8  FROM
9  (SELECT *, ROW_NUMBER() OVER (PARTITION BY company_name ORDER BY vendas DESC) AS listado
10 FROM
11     (SELECT date(timestamp) AS date, c.company_name, sum(amount) AS vendas
12     FROM transactions AS t
13     JOIN companies AS c
14     ON t.business_id = c.company_id
15     GROUP BY date(timestamp), c.company_id) AS rn) AS l
16
17 WHERE listado <=5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	date	company_name	vendas	listado
1	2021-09-25	A Institute	1206.24	1
2	2016-12-30	A Institute	948.32	2
3	2019-07-01	A Institute	890.42	3
4	2019-12-06	A Institute	873.70	4
5	2020-12-20	A Institute	782.64	5
6	2019-03-19	Ac Fermentum Incorporated	3149.04	1
7	2020-12-11	Ac Fermentum Incorporated	2717.76	2
8	2022-05-03	Ac Fermentum Incorporated	2714.44	3
9	2022-03-16	Ac Fermentum Incorporated	2701.45	4
10	2021-03-26	Ac Fermentum Incorporated	2462.04	5
11	2022-07-28	Ac Industries	1257.96	1
12	2018-06-27	Ac Industries	1165.41	2

Result 3

Action Output

#	Time	Action	Message
1	10:02:20	SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY company_name ORDER BY vendas DESC) AS listado FROM (...	500 row(s) returned

Calculé los 5 días que han generado más ingresos en cada empresa. Hice una join, agrupé por empresa y fecha y calculé el total de amount (por día y empresa). A continuación, con ROW_NUMBER, hice la partición por empresa y ordene por el total de ventas, de manera que me listó cada empresa por separado de mayor a menor ventas. Finalmente, aplico el filtro WHERE para mostrar solo los 5 primeros registros de cada empresa, es decir, los 5 días con más ventas.

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```sql
SELECT
    c.company_name,
    c.phone, c.country,
    date(timestamp) AS `date`,
    sum(amount) AS vendas
FROM transactions AS t
JOIN companies AS c
ON t.business_id = c.company_id
WHERE
    date (timestamp)= '2015-04-29'
    OR date (timestamp)= '2018-7-20'
    OR date (timestamp)= '2024-03-13'
AND declined = 0
GROUP BY c.company_id, `date`
HAVING vendas BETWEEN 350 AND 400
ORDER BY sum(amount) DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	company_name	phone	country	date	vendas
1	Auctor Mauris Vel LLP	08 09 28 74 14	United States	2018-07-20	399.51
2	Orci Adipiscing Limited	03 18 00 77 81	United Kingdom	2018-07-20	377.54
3	Fringilla LLC	08 29 15 93 57	New Zealand	2015-04-29	367.62
4	Pede Cum Ltd	07 62 26 48 38	Norway	2018-07-20	356.87
5	Auctor Mauris Vel LLP	08 09 28 74 14	United States	2024-03-13	353.75
6	Justo Eu Arcu Ltd	08 42 56 71 52	Italy	2024-03-13	350.82

Result 5

Action Output

#	Time	Action	Message
1	10:07:07	SELECT c.company_name, c.phone, c.country, date(timestamp) AS `date`, sum(amount) AS vendas FROM transactions AS t JOI...	6 row(s) returned

Muestro las empresas que realizaron transacciones en las fechas dadas y cuyo valor total asciende entre 350 y 400 euros. Es decir, agrupé por empresa y fecha y calcule el sumatorio del amount de transacciones de cada empresa en cada día. Finalmente, con HAVING, me quedo con aquellos valores comprendidos entre 350 y 400 euros.

Exercici 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen més de 400 transaccions o menys.

Primero hice una JOIN, agrupé por empresa y con count(*) calculé el total de transacciones de cada empresa. A continuación, usé CASE para establecer una columna nueva indicando si el número de transacciones está por encima o por debajo de 400 transacciones. Finalmente, utilicé ROW_NUMBER grupando por “por encima” o “por debajo” y ordene por el número de transacciones. Así me queda una lista de las empresas de cada grupo.

```sql
SELECT *, ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY num_transactions ASC) AS lista
FROM
    (SELECT *, CASE
            WHEN num_transactions < 400 THEN 'Por debajo'
            ELSE 'Por encima'
            END AS categoria
    FROM

            (SELECT
                c.company_id,
                c.company_name,
                count(*) AS num_transactions
            FROM transactions AS t
            JOIN companies AS c
            ON t.business_id = c.company_id
            WHERE declined = 0
            GROUP BY c.company_id, c.company_name) AS nt) AS nu;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	company_ic	company_name	num_transaction	categoria	lista
1	b-2362	Lorem Eu Incorporated	378	Por debajo	1
2	b-2286	Fringilla LLC	396	Por debajo	2
3	b-2470	Nec Luctus LLC	399	Por debajo	3
4	b-2578	Dui Quis Institute	400	Por encima	1
5	b-2226	Magna A Neque Industries	406	Por encima	2
6	b-2446	Risus Associates	407	Por encima	3
7	b-2526	Amet Luctus Vulputate Foundation	410	Por encima	4
8	b-2534	Mattis Foundation	412	Por encima	5
9	b-2590	Euismod Mauris Institute	417	Por encima	6

Result 7

Action Output

#	Time	Action	Message
1	10:15:45	SELECT *, ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY num_transactions ASC) AS lista FROM (SELECT *, CASE ...	100 row(s) returned

Exercici 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

```sql
DELETE FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Action Output

#	Time	Action	Message
1	10:19:44	DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected

Con DELETE y WHERE elimino las transacción dada.

Exercici 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

```sql
CREATE VIEW VistaMarketing AS
SELECT c.company_name, c.phone, c.country, ROUND(AVG(amount),2) AS media
FROM companies AS c
JOIN transactions AS t
ON c.company_id = t.business_id
GROUP BY c.company_id, c.company_name, c.phone, c.country
ORDER BY ROUND(AVG(amount),2) DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

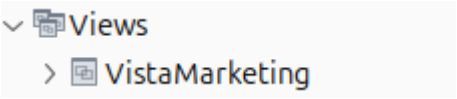
#	company_name	phone	country	media
1	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87
2	Pretium Neque Corp.	07 77 48 55 28	Australia	276.16
3	Urna Convallis Associates	06 01 24 77 04	United States	274.24
4	At Associates	09 56 61 10 65	New Zealand	272.21
5	Metus Vitae Associates	08 25 44 40 66	Australia	270.08
6	Aliquet Diam Limited	02 76 61 47 46	United States	269.60
7	Nec Luctus LLC	02 14 71 75 73	Norway	268.60
8	Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.85
9	Tortor Nunc Commodo Company	05 35 92 77 16	United States	267.84
10	Cras Consulting	07 50 10 85 63	Belgium	267.44
11	Eget Ipsum Ltd	03 67 44 56 72	United States	267.06
12	Sed LLC	01 63 16 26 52	Belgium	266.07
13	Amet Institute	06 33 40 21 33	Australia	265.54
14	Pede Ultrices Ltd	04 52 40 26 17	Belgium	265.36
15	Fusce Corp.	08 14 97 58 85	United States	265.23
16	Tincidunt Orci Limited	01 78 18 81 44	Norway	265.10
17	Donec Ltd	01 25 51 37 37	Norway	265.01
18	Vestibulum Lorem PC	02 02 87 33 40	Belgium	264.96
19	Orci Adipiscing Limited	03 18 00 77 81	United Kin...	264.79

Result 11

Action Output

#	Time	Action	Message
1	10:26:02	SELECT c.company_name, c.phone, c.country, ROUND(AVG(amount),2) AS media FROM companies AS c JOIN transactions AS t ON c.co...	100 row(s) returned

Creo la vista con CREATE VIEW AS. En la query hago una JOIN, agrupo por empresa, calculo la media de amount y muestro los campos que se piden.



3. Nivell 3

Nivell 3
Exercici 1
Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

Los pasos que seguí se describen a continuación:

- Agrupo por tarjeta de credito, ordeno el resultado por fecha empezando por las transacciones más recientes y creo un listado de todos los registros de cada targeta
- A continuación, con el filtro WHERE selecciono de la lista los 3 primeros registros de cada tarjeta de crédito, es decir, me quedo con las 3 transacciones más recientes de cada una
- Luego hice un sumatorio acumulativo con OVER de manera que sume el 'declined' de las 3 transacciones más recientes de cada tarjeta y coloque el resultado en una nueva columna llamada 'transacciones_declinadas'. El sumatorio se acumulava en la fila tercera de cada tarjeta, es decir, es la fila que nos interesa porque nos muestra las cantidad de transacciones declinadas
- Finalmente, establecí un condicional con CASE en función del valor de la fila 3; si este valor es = 3 → tarjeta inactiva / else → tarjeta activa. El estado de la tarjeta se refleja en otra columna llamada 'estado_targeta'
- A continuación, cree la tabla nueva (taula_nova)
- Para obtener el resultado a la pregunta, tenemos que hacer un SELECT (*) de esta tabla con el estado_targeta='activa';

```
14    (SELECT card_id,
15            CASE
16                WHEN transacciones_declinadas = '3' THEN 'inactiva'
17                ELSE 'activa'
18            END AS estado_targeta
19    FROM
20                (SELECT *, SUM(declined) OVER (PARTITION BY card_id ORDER BY lista) AS transacciones_declinadas
21                FROM
22                        (SELECT card_id, declined, date(timestamp) AS `date`,
23                        ROW_NUMBER() OVER(PARTITION BY card_id ORDER BY date(timestamp) DESC) AS lista
24                        FROM transactions) AS ls
25                WHERE lista BETWEEN '1' AND '3') AS suma_acumulativa
26    WHERE lista = '3') tabla_final;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

#	card_id	estado_targeta
1	CcS-4857	activa
2	CcS-4858	activa
3	CcS-4859	activa
4	CcS-4860	activa
5	CcS-4861	activa
6	CcS-4862	activa
7	CcS-4863	activa
8	CcS-4864	activa
9	CcS-4865	activa
10	CcS-4866	activa
11	CcS-4867	activa

Result 91

Action Output

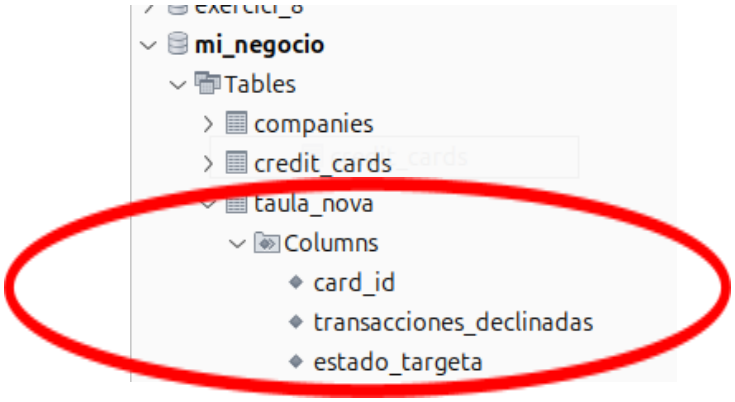
#	Time	Action	Message
1	17:47:52	SELECT card_id, CASE WHEN transacciones_declinadas = '3' THEN 'inactiva' ELSE 'activa' END ...	5000 row(s) returned

```sql
CREATE TABLE taula_nova AS
SELECT *
FROM

    (SELECT card_id,
            CASE
                WHEN transacciones_declinadas = '3' THEN 'inactiva'
                ELSE 'activa'
            END AS estado_targeta
    FROM
                (SELECT *, SUM(declined) OVER (PARTITION BY card_id ORDER BY lista) AS transacciones_declinadas
                FROM
                        (SELECT card_id, declined, date(timestamp) AS `date`,
                        ROW_NUMBER() OVER(PARTITION BY card_id ORDER BY date(timestamp) DESC) AS lista
                        FROM transactions) AS ls
                WHERE lista BETWEEN '1' AND '3') AS suma_acumulativa
    WHERE lista = '3') tabla_final;
```

Action Output

#	Time	Action	Message
1	17:48:45	CREATE TABLE taula_nova AS SELECT * FROM (SELECT card_id, CASE WHEN transacciones_declinadas = '3' THEN 'i...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0



👉 Quantes targetes estan actives?

```sql
SELECT count(*)
FROM taula_nova
WHERE estado_targeta = 'activa';
```

Result Grid Filter Rows: Export: Wrap Cell Content:

#	count(*)
1	4995

Result 41

Action Output

#	Time	Action	Message
1	10:59:35	SELECT count(*) FROM taula_nova WHERE estado_targeta = 'activa'	1 row(s) returned

Exercici 2

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Primero creo la tabla.

```
45 • CREATE TABLE products(
46   id INT NOT NULL,
47   product_name VARCHAR (30),
48   price VARCHAR(23),
49   colour VARCHAR (12),
50   weight FLOAT,
51   warehouse_id VARCHAR (9),
52   PRIMARY KEY (id));
53
54   -- importo les dades desde el terminal
```

Action Output

#	Time	Action	Message
1	11:05:02	CREATE TABLE products(id INT NOT NULL, product_name VARCHAR (30), price VARCHAR(23), colour VARCHAR (12), weight FLOAT, wa...	0 row(s) affected

Hago una primera observación del archivo desde mi terminal. Observo los diferentes campos.

```
==> N3.Ex.2__ products.csv <==
id,product_name,price,colour,weight,warehouse_id
1,Direwolf Stannis,$161.11,#7c7c7c,1,WH-4
2,Tarly Stark,$9.24,#919191,2,WH-3
3,duel tourney Lannister,$171.13,#d8d8d8,1.5,WH-2
4,warden south duel,$71.89,#111111,3,WH-1
5,skywalker ewok,$171.22,#dbdbdb,3.2,WH-0
6,dooku solo,$136.60,#c4c4c4,0.8,WH--1
7,north of Casterly,$63.33,#b7b7b7,0.6,WH--2
8,Winterfell,$32.37,#383838,1.4,WH--3
9,Winterfell,$76.40,#b5b5b5,1.2,WH--4
```

Accedo a la base de datos mi negocio configurando infile=1.

```
>  mysql --local-infile=1 -u root -p mi_negocio
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 89
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Finalmente, cargo los datos a la tabla y compruebo que se cargaron bien.

```
mysql> LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N3.Ex.2__ products.csv'
    -> INTO TABLE products
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\n'
    -> IGNORE 1 ROWS;
Query OK, 100 rows affected (0,02 sec)
Records: 100  Deleted: 0  Skipped: 0  Warnings: 0

mysql>
```

```
64 •  SELECT * FROM products;
65
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

#	id	product_name	price	colour	weight	warehouse_ic
1	1	Direwolf Stannis	$161.11	#7c7c7c	1	WH-4
2	2	Tarly Stark	$9.24	#919191	2	WH-3
3	3	duel tourney Lannister	$171.13	#d8d8d8	1.5	WH-2
4	4	warden south duel	$71.89	#111111	3	WH-1
5	5	skywalker ewok	$171.22	#dbdbdb	3.2	WH-0
6	6	dooku solo	$136.60	#c4c4c4	0.8	WH--1
7	7	north of Casterly	$63.33	#b7b7b7	0.6	WH--2
8	8	Winterfell	$32.37	#383838	1.4	WH--3
9	9	Winterfell	$76.40	#b5b5b5	1.2	WH--4
10	10	Karstark Dorne	$119.52	#f4f4f4	2.4	WH--5
11	11	Karstark Dorne	$49.70	#141414	2.7	WH--6

products 44

Action Output

#	Time	Action	Message
1	11:18:25	SELECT * FROM products	100 row(s) returned

Como se observa en la imagen anterior, el campo 'price' aparece con el símbolo ($) en todos los registros. Como esto puede dificultar cálculos posteriores, se decidió por quitar este símbolo.

```
62
63 •  UPDATE products
64    SET price =
65    REPLACE (price, '$', '')
66    LIMIT 500;
67
```

Action Output

#	Time	Action	Message
1	18:27:12	UPDATE products SET price = REPLACE (price, '$', '') LIMIT 500	100 row(s) affected Rows matched: 100 Changed: 100 Warnings: 0

Aquí comprobé que efectivamente se lleva a cabo la sustitución.

```
69 •  SELECT price FROM products;
70
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

#	price
1	161.11
2	9.24
3	171.13
4	71.89
5	171.22
6	136.60
7	63.33
8	32.37
9	76.40
10	119.52
11	49.70

products 104

Action Output

#	Time	Action	Message
1	18:35:24	SELECT price FROM products LIMIT 0, 300	100 row(s) returned

Luego modifiqué el tipo de columna a FLOAT, ya que inicialmente la clasificamos como VARCHAR para poder cargar los datos sin problemas.

```
71 • ALTER TABLE products
72   MODIFY COLUMN price FLOAT;
```

Action Output

#	Time	Action	Message
1	18:39:30	ALTER TABLE products MODIFY COLUMN price FLOAT	100 row(s) affected Records: 100 Duplicates: 0 Warning

Table: products

Columns:
id int PK
product_name varchar(30)
price float
colour varchar(12)
weight float
warehouse_id varchar(9)

A la hora de hacer el diagrama, resulta que la relación entre la tabla products y transactions es n:n, por lo que hace falta una tabla intermedia.

Primero creo la tabla con una PK nueva y con autoincrement, una columna que es product_id y otra transactions_id, de manera que cada registro representa una transacción con un producto.

```
69 • CREATE TABLE transactions_products (
70       id INT NOT NULL AUTO_INCREMENT,
71       transactions_id VARCHAR(255),
72       product_id INT,
73       PRIMARY KEY (id));
74
```

Action Output

#	Time	Action	Message
1	11:41:21	CREATE TABLE transactions_products (id INT NOT NULL AUTO_INCREMENT, transactions_id VARCHAR(255), product_id INT, PRI...	0 row(s) affected

Antes de rellenar la tabla nueva con los datos, experimenté haciendo queries hasta obtener el resultado deseado: expandir las transacciones en filas en función de los productos asociados. En la siguiente imagen observamos la query que quiero usar para rellenar la tabla nueva.

```
78 •  SELECT id, productes_per_client
79    FROM transactions,
80    JSON_TABLE (
81        CONCAT('["', REPLACE (product_ids, ',' , '","'), '"]'),
82        '$[*]' COLUMNS (
83                        productes_per_client INT PATH "$")) AS t;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

#	id	productes_per_clie
1	00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
2	00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
3	00043A49-2949-494B-A5DD-A5BAE3BB19DD	97
4	00043A49-2949-494B-A5DD-A5BAE3BB19DD	87
5	00045D6B-ED2E-4F2F-8186-CEE074D875D0	30
6	00045D6B-ED2E-4F2F-8186-CEE074D875D0	11
7	00045D6B-ED2E-4F2F-8186-CEE074D875D0	16
8	00045D6B-ED2E-4F2F-8186-CEE074D875D0	81
9	000481C3-1C26-4FEF-83A0-4CD0EB004BBD	72
10	00051AA4-9CBE-4268-B070-C38062A1B3E2	18
11	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	35
12	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	33
13	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	19
14	0009A151-9BCF-4E31-9053-A468FF77FAAB	93
15	0009A151-9BCF-4E31-9053-A468FF77FAAB	55
16	0009A151-9BCF-4E31-9053-A468FF77FAAB	28
17	0009A151-9BCF-4E31-9053-A468FF77FAAB	91
18	0009D494-6245-4DF9-955D-2C084191CFFB	55
19	0009D494-6245-4DF9-955D-2C084191CFFB	8
20	0009D494-6245-4DF9-955D-2C084191CFFB	72

Result 58

Action Output

#	Time	Action	Message
1	12:52:46	SELECT id, productes_per_client FROM transactions, JSON_TABLE (CONCAT('["', REPLACE (product_ids, ',' , '","'), '"]'), '$[*]' COLUMN...	253388 row(s) returned

Usé la función JSON_TABLE para dicho fin. Me costó un poco porque nunca había trabajado anteriormente con estas funciones JSON. Lo que hice en el INSERT fue insertar en la columna transactions_id las transacciones y el la product_id los productos. Con JSON_TABLE, conseguimos modificar el formato de la columna, convirtiéndolo en un array de strings de formato JSON. Para ello, usamos la función CONCAT con [ ], y la función REPLACE que nos va a sustituir la coma de entre los números por “,”. Es decir hacemos la siguiente transformación: 1,2,3 → [“1”, “2”, “3”] para cada uno de los registros.

A continuación, le añadimos el path, que en este caso es: ‘’$[*]. Estos nos sirve para generar filas de datos relacionados. Es decir, le estamos diciendo que nos haga tantas filas como valores aparecen en la fila original. En columns, le decimos la columna donde queremos que se añadan las filas expandidas, el tipo de dato y el path, indicando que encuentre los valores JSON y que me los devuelva expandidos en formato sql en filas individuales. También hay que añadirle un alias final.

```
88 •  INSERT INTO transactions_products(transactions_id, product_id)
89       (SELECT id AS transactions_id, product_id
90       FROM transactions,
91       JSON_TABLE (
92           CONCAT('["', REPLACE (product_ids, ',' , '","'), '"]'),
93           '$[*]' COLUMNS (
94                           product_id INT PATH "$")) AS t);
```

Action Output

#	Time	Action	Message
1	12:49:15	INSERT INTO transactions_products(transactions_id, product_id) (SELECT id AS transactions_id, product_id FROM transactions, J...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0

En la siguiente imagen compruebo que se haya realizado con éxito la inserción de los datos.

```
97 •  SELECT * FROM transactions_products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

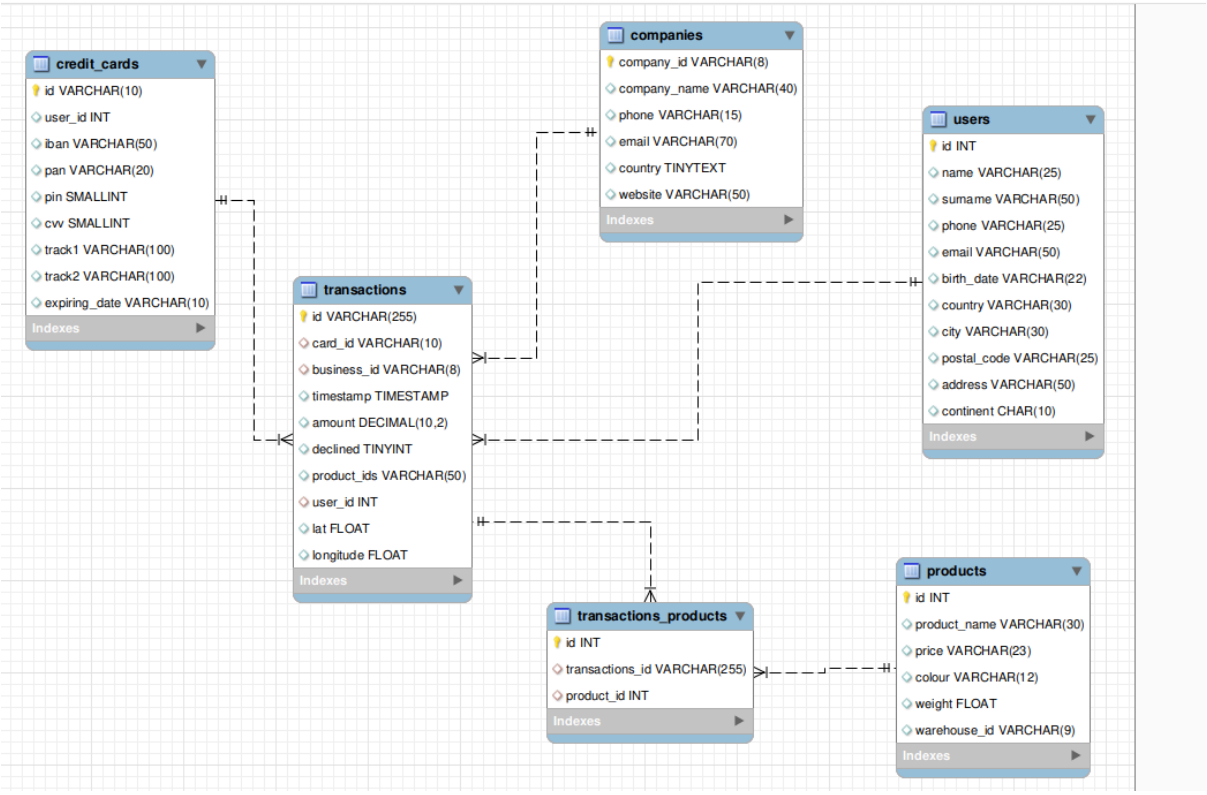
#	id	transactions_id	product_id
1	1	00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
2	2	00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
3	3	00043A49-2949-494B-A5DD-A5BAE3BB19DD	97
4	4	00043A49-2949-494B-A5DD-A5BAE3BB19DD	87
5	5	00045D6B-ED2E-4F2F-8186-CEE074D875D0	30
6	6	00045D6B-ED2E-4F2F-8186-CEE074D875D0	11
7	7	00045D6B-ED2E-4F2F-8186-CEE074D875D0	16
8	8	00045D6B-ED2E-4F2F-8186-CEE074D875D0	81
9	9	000481C3-1C26-4FEF-83A0-4CD0EB004BBD	72
10	10	00051AA4-9CBE-4268-B070-C38062A1B3E2	18
11	11	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	35
12	12	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	33
13	13	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	19
14	14	0009A151-9BCF-4E31-9053-A468FF77FAAB	93
15	15	0009A151-9BCF-4E31-9053-A468FF77FAAB	55
16	16	0009A151-9BCF-4E31-9053-A468FF77FAAB	28
17	17	0009A151-9BCF-4E31-9053-A468FF77FAAB	91
18	18	0009D494-6245-4DF9-955D-2C084191CFFB	55
19	19	0009D494-6245-4DF9-955D-2C084191CFFB	8
20	20	0009D494-6245-4DF9-955D-2C084191CFFB	72
21	21	000A1DEC-CDB6-4AB2-A619-71DAB8D4A262	46
22	22	000A1DEC-CDB6-4AB2-A619-71DAB8D4A262	56
23	23	000A1DEC-CDB6-4AB2-A619-71DAB8D4A262	73
24	24	000A1E64-1414-40B0-9D92-5678A4D958E2	6
25	25	000A1E64-1414-40B0-9D92-5678A4D958E2	89
26	26	000A1E64-1414-40B0-9D92-5678A4D958E2	19

transactions_products 56

Action Output

#	Time	Action	Message
1	12:50:26	SELECT * FROM transactions_products	253388 row(s) returned

El esquema final queda como a continuación se muestra, con una nueva tabla de hechos. Realmente son dos las tablas de hechos: transactions y transactions_products. Pero considero que la transactions_products es la tabla de hechos principal pues quedan reflejadas exclusivamente las transacciones, mientras que la tabla transactions sería más secundária, pues aparte de las transacciones, aporta otra información adicional.



👉 Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```
103 •  SELECT p.id, p.product_name, count(*) AS num_ventas
104    FROM transactions_products tp
105    JOIN products p
106    ON tp.product_id = p.id
107    GROUP BY p.id
108    ORDER BY count(*) DESC;
```

Result Grid Filter Rows: Export: Wrap Cell Content:

#	id	product_name	num_ventas
1	52	riverlands the duel	2654
2	29	Tully maester Tarly	2635
3	21	duel Direwolf	2609
4	16	the duel warden	2608
5	66	mustafar jinn	2600
6	48	rock Renly in	2597
7	33	duel warden	2597
8	87	sith Jade	2597
9	23	riverlands north	2593
10	68	Stark Karstark	2589
11	88	Stannis warden ...	2587
12	4	warden south duel	2584
13	28	chewbacca must...	2584
14	73	Dorne bastard	2584
15	34	of north	2580
16	31	Lannister	2580
17	38	ewok sith	2578

Result 73

Action Output

#	Time	Action	Message
1	13:39:40	SELECT p.id, p.product_name, count(*) AS num_ventas FROM transactions_products tp JOIN products p ON tp.product_id = p.id GROUP...	100 row(s) returned

Hago una JOIN entre productos y transactions_products, agrupo por id del producto y cuento los registros obteniendo así el número de ventas de cada producto.